

# MiniTPSOR



**Materia:** Sistemas Operativos y Redes

**Docentes:**

- Giuliana Zorzoli
- Augusto Liali

**Alumnos:**

- Gaston Rodriguez
- Facundo Palavecino
- Nicolas Contreras

## EJERCICIO 1: Shell y terminal

```
INICIO miniTP.sh

SI no se recibe parámetro ENTONCES
    mostrar "Error: debe ejecutar con un parámetro al menos"
    terminar con error
FIN SI

NOMBRE_DIR ← primer parámetro recibido
RUTA_DESTINO ← HOME / NOMBRE_DIR
ARCHIVO_SALIDA ← RUTA_DESTINO / contenido_home.txt

crear directorio RUTA_DESTINO (si ya existe, no lanzar error)

listar todos los archivos del HOME (incluidos ocultos, con
permisos)
guardar ese listado en ARCHIVO_SALIDA

mostrar por pantalla el contenido de ARCHIVO_SALIDA
esperar que el usuario presione ENTER
terminar correctamente

FIN miniTP.sh
```

## EJERCICIO 2: Estados de un Proceso

El programa en C alterna entre operaciones de I/O y cálculos aritméticos para forzar cambios de estado visibles en htop:

```
INICIO programa estados_proceso

    mostrar mensaje de inicio al usuario

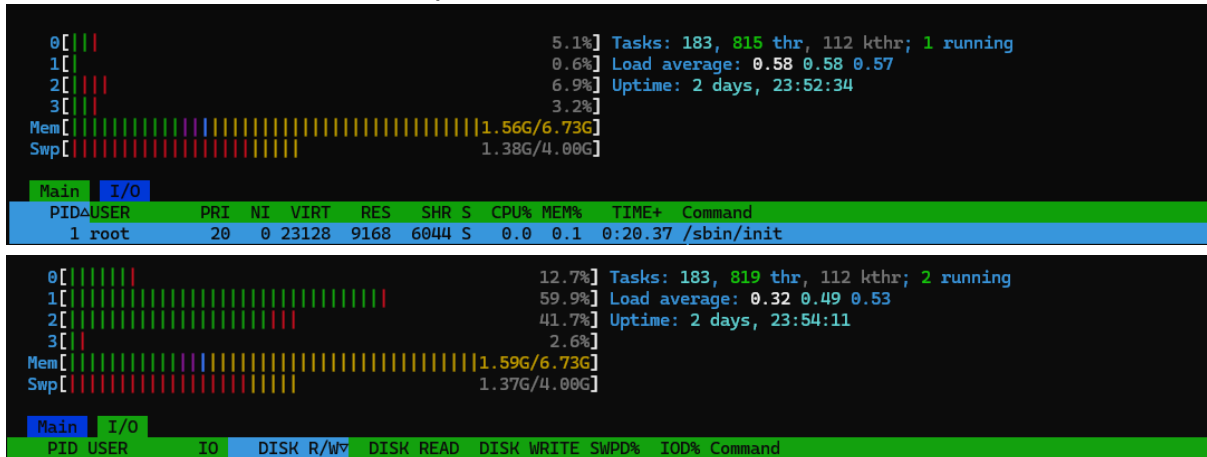
    — FASE 1: I/O Bound (estado S en htop) —
    mostrar "Ingresa un número para comenzar"
    leer entrada del usuario          ← proceso entra en estado S
    (Sleeping)

    — FASE 2: CPU Bound (estado R en htop) —
    PARA i desde 0 hasta 2.000.000.000 HACER
        resultado ← resultado + (i * 0.0001)    ← operación aritmética
    FIN PARA
        ← proceso en estado R (Running) durante este
    loop

    mostrar resultado del cálculo
    esperar que el usuario presione ENTER

FIN programa estados_proceso
```

Resultados observados en htop:



Durante la espera de input el proceso aparece en estado S (Interruptible Sleep), con uso de CPU en 0%. Una vez que el usuario ingresa el número y comienza el loop aritmético, el proceso pasa a estado R (Running) y el uso de CPU sube notablemente.

## EJERCICIO 3: Threads

Se implementaron dos versiones del programa: una secuencial (sin hilos) y una paralela (con 5 hilos usando pthreads).

### Versión sin hilos:

```
INICIO programa secuencial
  registrar tiempo de inicio

  llamar a calculo_aritmetico() ← ejecución 1
  llamar a calculo_aritmetico() ← ejecución 2
  llamar a calculo_aritmetico() ← ejecución 3
  llamar a calculo_aritmetico() ← ejecución 4
  llamar a calculo_aritmetico() ← ejecución 5

  registrar tiempo de fin
  mostrar tiempo total transcurrido
FIN programa secuencial
```

```
FUNCIÓN calculo_aritmetico()
  contador ← 0
  MIENTRAS contador < 2.147.483.647 HACER
    contador ← contador + 1
  FIN MIENTRAS
FIN FUNCIÓN
```

### Versión con hilos (paralela):

```
INICIO programa con hilos
  registrar tiempo de inicio

  PARA i desde 0 hasta 4 HACER
```

```
        crear hilo[i] que ejecute calculo_aritmetico()
    FIN PARA    ← los 5 hilos corren en paralelo

    PARA i desde 0 hasta 4 HACER
        esperar a que hilo[i] termine (join)
    FIN PARA

    registrar tiempo de fin
    mostrar tiempo total transcurrido
FIN programa con hilos
```

### Medición de tiempos y análisis:

Al ejecutar la versión sin hilos con time, los resultados obtenidos fueron:

```
real    0m0.064s
user    0m0.030s
sys     0m0.000s
```

Con la versión paralela (5 hilos), el tiempo "real" es significativamente menor, ya que todas las llamadas a `calculo_aritmetico()` se ejecutan de forma simultánea en distintos núcleos del procesador, en lugar de hacerlo una después de la otra.

La cantidad de núcleos es clave porque determina cuántos hilos pueden correr verdaderamente en paralelo. En este equipo, la CPU cuenta con 6 núcleos físicos y 12 procesadores lógicos (hyper-threading), lo que permite que los 5 hilos corran al mismo tiempo sin necesidad de turnarse el procesador.